Temporal Olympic Medal Prediction with Performance Momentum

UE24CS352A Machine Learning mini-project, Team 53. Ryona Aditree Ghosh (PES1UG24AM233) and Rishit V N S (PES1UG24AM225). Code: github.com/ryonahh/ML_MiniProject_53

1. Problem statement

The reference paper (Dobkowski, *2020 Summer Olympics Predictions Using Machine Learning*) predicts each country's medal count by combining World Bank indicators with the previous Games' medals, comparing many regressors and a two-stage classifier-then-regressor design. We reproduce that setup and ask: **using only information available before a Games, does explicitly modelling a country's performance momentum improve medal forecasts?** Last Games' count is already a strong forecast, so any gain must come from how a country's trajectory is represented.

2. Dataset

We use Kaggle's *120 years of Olympic history* (athlete_events.csv, 271,116 athlete-event rows; noc_regions.csv). We keep Summer Games only and drop the unofficial 1906 Games. The raw file repeats a team medal once per athlete, so medals are de-duplicated per (NOC, Games, event, medal), which reproduces the official tables (2016: USA 121, China 70, UK 67). NOC codes are mapped to regions so historical teams form one history (FRG/GDR to Germany, URS/EUN to Russia). The result is 2,770 Country × Games rows; the target is *Total_Medals*. As in the paper, we add six World Bank WDI series (GDP, GDP per capita, GDP growth, population, population growth, land area) plus shares of world GDP and population, always from the year *before* the Games. The World Bank has no data for the USSR, East Germany or Taiwan, so 8% of rows lack these features (Random Forests handle missing values natively; linear models impute inside their pipelines). The real Tokyo 2020 medal table is used only to score our 2020 forecast.

3. Approach

Leakage-free features. Every feature for Games *t* comes from the country's earlier appearances (group-wise shift): previous medals and golds; previous athletes, sports and events; medal growth and athlete growth as log-differences between the last two Games; a three-Games rolling average; and medal efficiency (medals per athlete).

Novelty: Performance Momentum. We test momentum in two ways. (i) A *momentum score*: the mean of the z-scored medal growth, athlete growth and efficiency, with the scaler fitted inside the sklearn pipeline on training Games only. (ii) A *momentum target* (our proposal): the Random Forest learns $\log(1+\text{medals}_t) - \log(1+\text{medals}_{t-1})$ and the forecast is $(1+\text{medals}_{t-1}) \cdot e^{\text{pred}} - 1$. A forest can only output averages of targets it has seen, so learning counts pulls large nations toward the middle; learning change lets one leaf serve a 3-medal and a 90-medal nation on the same trajectory.

Controlled comparison. Eight models share one Random Forest (300 trees, min_samples_leaf=10, max_features=0.6), chosen once by mean validation MAE on 2012 across all models, so only features and targets differ: Baseline (previous medals and golds), Historical Features, + Momentum Score, + Economic, the paper's Two-Stage design on history + World Bank inputs, and Momentum Target alone, + Score and + Economic. Separately, we re-run the paper's model families (Linear, Ridge, Lasso, Poisson, SVR, Random Forest; and Logistic, SVM, Naive Bayes, MLP, Random Forest classifiers), each tuned on 2012 and tested on 2016, with both targets. We train on 1964–2012 and test on Rio 2016 (five seeds), backtest every Games from 1996 to 2016 with an expanding window, run a paired bootstrap on the 2016 errors, and forecast Tokyo 2020 from data up to 2016.

4. Implementation

A Python package (`src/olympic_momentum`): `data.py` (download, clean, aggregate), `economic.py` (lagged World Bank join), `features.py` (lagged features and the `PerformanceMomentum` transformer), `models.py` (a `MedalForecaster` that wraps the pipeline and the target encoding, plus the two-stage model), `experiment.py` (tuning, test, backtest, bootstrap, forecast), `model_zoo.py` (the paper's model families) and `reporting.py`. One command, `python scripts/run_pipeline.py`, regenerates every table, figure and the website data. Sixteen pytest tests check team-medal counting, that changing a future result never alters past features, that World Bank values come from the year before, and that every model fits and predicts non-negative counts. A demo notebook and an interactive website (docs/index.html) present the results.

5. Results

On the held-out Rio 2016 Games the Momentum Target model reaches **MAE 1.298**, RMSE 3.37 and R^2 0.933: **19.2%** lower MAE than the baseline and **11.9%** lower than the same nine features predicting the count. The paired bootstrap gives a 95% interval of 0.02 to 0.34 fewer medals of error per country (99% of resamples favour our model). In the backtest a momentum-target model has the lowest error in all six Games, 1996–2016.

Model	MAE	RMSE	R^2	Top-10 MAE	Backtest MAE
Baseline	1.606	3.85	0.913	12.26	1.629
Historical Features	1.474	3.76	0.916	11.91	1.599
Historical + Momentum Score	1.501	3.81	0.914	12.26	1.618
Historical + Economic	1.461	3.68	0.920	11.07	1.634
Paper Two-Stage	1.442	3.75	0.917	11.34	1.557
Momentum Target	**1.298**	**3.37**	**0.933**	**10.14**	**1.374**
Momentum Target + Score	1.296	3.39	0.932	10.16	1.389
Momentum Target + Economic	1.307	3.38	0.932	9.83	1.387

Table 1. Rio 2016 held-out test (mean of five seeds) and average MAE over the six-Games backtest. Lower is better except R^2.

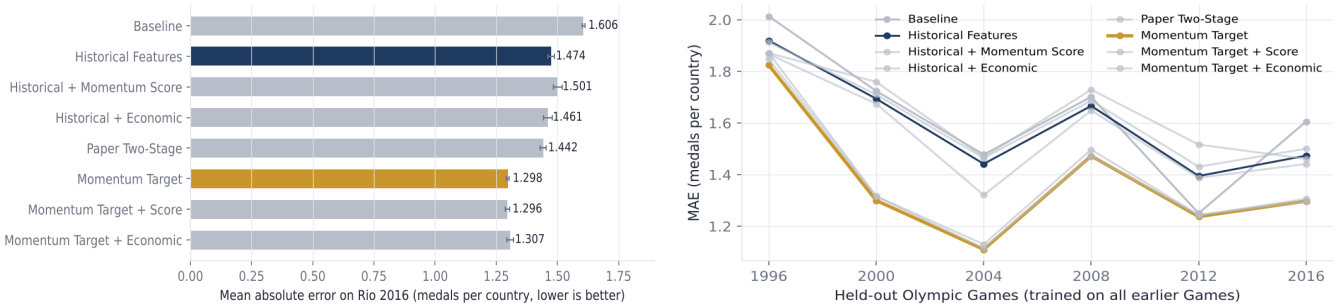


Figure 1. Left: Rio 2016 MAE per model (gold = proposed, navy = historical features). Right: MAE in each backtest Games.

Across model families (Rio 2016 MAE, count → momentum target): Linear 2.22 → 1.38, Ridge 2.20 → 1.35, Lasso 2.21 → 1.31, Random Forest 1.48 → 1.31; Poisson (count only) 1.66. SVR is the exception (1.32 → 1.58); its count version also has the best RMSE and top-10 error, on this single test year. The medal / no-medal classifiers reach 86.3–89.2% on 2012, matching the paper's 85.5–89.1%, but only a few points above the rule "medalled last time" (87.2%).

Some results did not support our expectations, and we report them as found. Adding the momentum *score* as a feature does not help (MAE 1.501 vs 1.474): the forest already has its three components. World Bank data barely changes the 2016 error (1.461). On the genuinely unseen Tokyo 2020 Games, however, the World Bank models do best: the paper's two-stage design has the lowest MAE (1.506) and Historical + Economic the best top-10 error (9.5), while Momentum Target is mid-pack (1.557; baseline 1.851). Tokyo was postponed by COVID and hosted by Japan (58 medals, up from 41).

6. Conclusions

How momentum is represented matters more than adding it. Turning it into the learning target (predicting each country's change rather than its count) gives a consistent and statistically supported improvement over the count model and the reference paper's two-stage design on 1996–2016, and it helps four of the five model families that can use it, while the composite score alone adds nothing. Tokyo 2020 shows the other side: when a cycle is disrupted, the paper's World Bank inputs carry signal that last Games' medals miss. Future work: host-nation flags, combining the momentum target with economic data in tuned SVR or boosting models, count-aware gradient boosting (Poisson/Tweedie), and event-level models.